CS & IT ENGINEERING



Algorithms

Dynamic Programming



Lecture No.- 04

By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Bellman Ford

0/1 Knapsack

Topics to be Covered



- Topic
- Topic
- Topic

Floyd Warshall

0/1 knapsack

LCS



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Dynamic Programming

Floyd Warshall Algorithm:

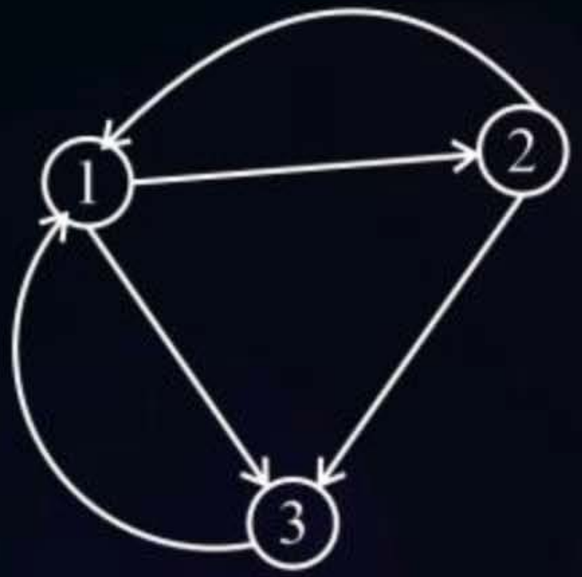
- For APSP (All Pairs Shortest Paths)



Topic : Dynamic Programming

Example:

Given, Cost adj matrix



C	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0





Topic : Dynamic Programming

DP based idea:

- Let $A^k(i, j)$ represent the cost of the path from a vertex 'i' (source) to some vertex 'j' (destination) with k being the highest intermediate vertex along the path from $i \rightarrow j$.



Topic : Dynamic Programming

$$A^k(i,j) = \min\{A^{k-1}(i,k) + A^{k-1}(k,j), A^{k-1}(i,j)\}$$

$1 \le k \le n$

$A^0(i,j) = C(i,j)$



↓

Initialization



Topic : Dynamic Programming

Tabulation

```
Algo FloydWarshall (G, C, n, e)
{
    A[1...n, 1...n]
    for (i=1; i<=n; i++)
        for (j=1; j<=n; j++)
            A[i, j] = C[i, j]
    for (k=1; k<=n; k++)
        for (i=1; i<=n; i++)
            for (j=1; j<=n; j++)
                A[i, j] = min(A[i,j], A[i, k] + A[k, j])
}
```

SC: $O(n^2)$ (Table (Additional Space))

initialization $\rightarrow O(n^2)$

$O(n^3)$



Topic : Dynamic Programming

Time and Space Complexity Analysis: Floyd Warshall

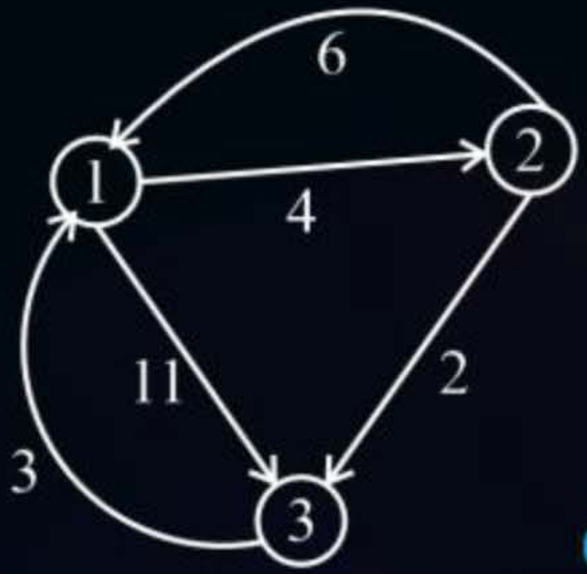
1) Time Complexity = $O(n^3)$

2) Space Complexity = $O(n^2)$ → Table A[n×n]

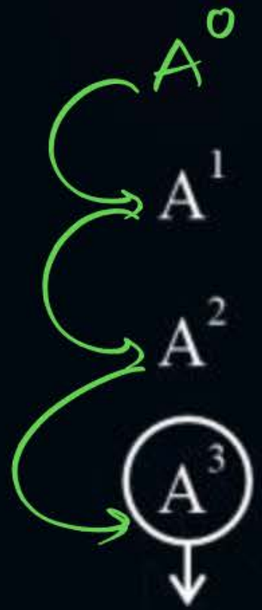


Topic : Dynamic Programming

Code Walkthrough:



n = 3



Tabulation

$A^0 \rightarrow A^1 \rightarrow A^2 \rightarrow A^3$

Initial

Final

C	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0



Topic : Dynamic Programming

Initialization, $A^0 = C$

A^0	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

Intermediate, k = 1

A^1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

k: 1 → n

k = 1

for(i: 1 → n

j: 1 → n

A(i, j)

Explanation:

$3 \rightarrow 2 : A^0 : \infty$

$3 \rightarrow 1 \rightarrow 2$

$A^0[3, 1] + A^0[1, 2] = 3 + 4 = 7$



Topic : Dynamic Programming

k: 1 → n

k = 2

A^2	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

Explanation:

$1 \rightarrow 3 : A^1 : 11$

$1 \rightarrow 2 \rightarrow 3$

$4 + 2 = 6$

k = 3

A^3	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0

Final APSP

Explanation:

$2 \rightarrow 1 : 6$

$2 \rightarrow 3 \rightarrow 1$

$A^2[2, 3] + A^2[3, 1] = 2 + 3 = 5$

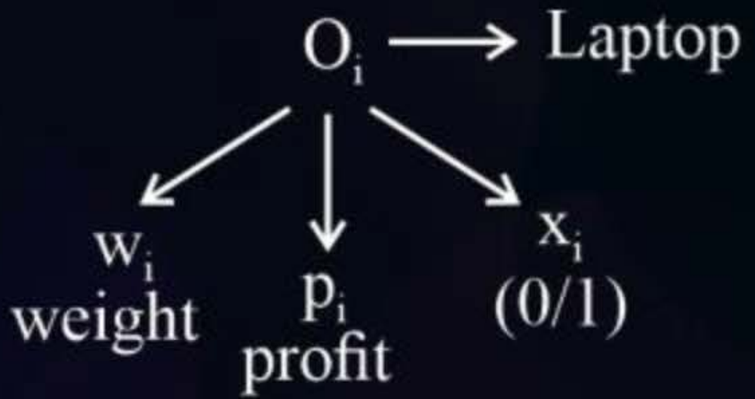


Topic : Dynamic Programming

0/1 Knapsack (Binary Knapsack):

(details in prev lec)

- Knapsack Capacity : M
- no. of objects → n





Topic : Dynamic Programming

Objective Function:

$$\max \sum_{i=1}^{n} p_i * x_i$$

Such that:

$$\Sigma w_i * x_i \leq M$$

$$x_i = \{0, 1\}$$



Topic : Dynamic Programming

Dynamic Programming Approach:

- Let 01 Knap(n, M) represent profit with n-objects and Knapsack of Capacity M.



Topic : Dynamic Programming

Recurrence:

$$01\ Knap(n, M) = 01\ Knap(n-1, M),\ w_n > M \quad \rightarrow \text{Exclude } O_n \text{ (no option)}$$

$$01\ Knap(n, M) = \max \begin{cases} 01\ Knap(n-1, M) \\ 01\ Knap(n-1, M-w_n) + p_n \end{cases} \quad \text{if } w_i \leq M$$

Initialization,

$$01\ Knap(n, M) = 0, \text{ if } n = 0 \text{ or } M = 0$$



Topic : Dynamic Programming

Example:

To understand if there are overlapping Sub-problem or not?

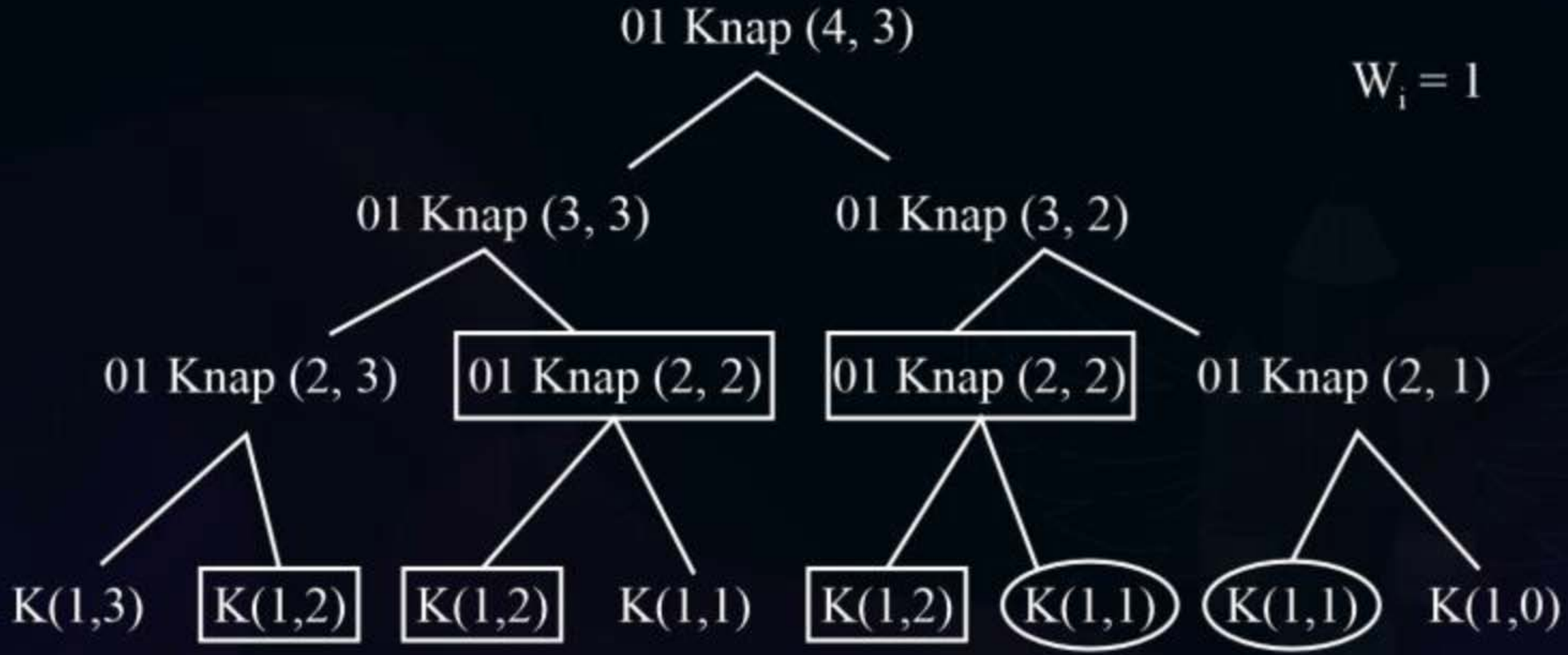
n = 4, M = 3

	01	02	03	04
P_i profits	10	20	30	40
W_i weights	1	1	1	1



Topic : Dynamic Programming

Top-Down (Recursive):





Topic : Dynamic Programming

Bottom-up Approach (Tabulation):

Example:

n = 4, M = 8

	01	02	03	04
P_i	1	2	3	6
W_i	2	3	4	5

Tabulation:

Let X[0...n, 0...M] be an array of which X[n, M] ⇒ final answer (max profit)



Topic : Dynamic Programming

P_i	W_i
1	2
2	3
5	4
6	5

m

X	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0	0	1	2	5	6	6	7	8

n

m = 8

7, 6, 8

X[3, 4]



Topic : Dynamic Programming

$X[i, j] = \max\{X[i-1, j], X[i-1, j-W_i] + P_i\}$

$X[2, 3] = \max\{X[1, 3], X[1, 3-3] + 2\}$

$X[2, 3] = \max\{1, 0+2\} = \max[1, 2] = 2$

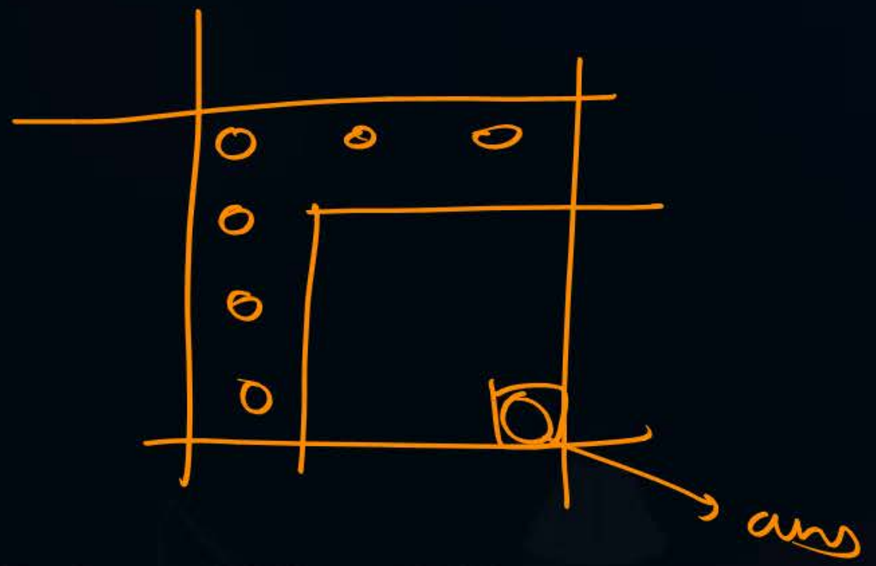
$X[2, 5]$ = max profit we can have with considering only 1st two objects and knapsack of capacity 5.



Topic : 0/1 Knapsack Code

```
Algo 01Knap(M, n, W, P) {
        X[0...n, 0...M]
        for (i = 0; i <=n; i++) {
                for (j = 0; j <= M; j++) {
                        if (i == 0 or j == 0) {
                                X[i, j] = 0
                        } else if (W[i] ≤ j) {
                                X[i, j] = max(X[i–1, j], X[i–1, j–W[i]] + P[i])
                        } else {
                                X[i, j] = X[i–1, j]
                        }
                }
        }
        return X[n, M]
}
```

(wi>j)





Topic : 0/1 Knapsack Code

X	0	1	2	.	.	.	M
0							
.							
.							
.							
n							

X[n, M]



Topic : 0/1 Knapsack Code

#Q. n = 3, M = 6

0/1 Knapsack

	01	02	03
P_i	1	2	5
W_i	2	3	4

Ans: 6

H.W



Topic : Dynamic Programming

Complexity Analysis of 0/1 Knapsack:

1. Time Complexity:
   - $O(n * M)$
2. Space Complexity:
   - $O((n+1) * (M+1)) = O(n * M)$

LCS



Topic : Dynamic Programming

Longest Common Sub-Sequence (LCS):

- Based on Strings
- A sequence/group/array of one or more characters.



Topic : Dynamic Programming

String:

1) Substring:

- A contiguous sequence of 1 or more characters from the given string.

abc



Topic : Dynamic Programming

String:

2) Subsequence:

- A sequence of 1 or more characters from the given string, that may or may not be contiguous, but their relative order is maintained.



Topic : Dynamic Programming

Example:

- String: "ABC" → length = 3
- Sub-strings: A, B, C → 3

  AB, BC → 2

  ABC → 1

No. of substrings = 3 + 2 +1 = 6



Topic : Dynamic Programming

Important Result:

Given : S[1, ..., n]

#Q. What are the total number of sub-strings of at-least length 1 (non-empty sub-strings)?

→ $\frac{n(n+1)}{2}$

$S[1 \ldots n]$

$l = 1, \longrightarrow n$

$l = 2, \longrightarrow (n-1)$

$l = 3 \longrightarrow (n-2)$

$\vdots$

$l = n \longrightarrow 1$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$



Topic : Dynamic Programming

Sub-Sequence:

- String = "ABC"
- Sub-Sequence = A, B, C, AB, BC, AC, ABC → Valid sub-sequence
- CA, CB → relative order not maintained, not a sub-sequence.



Topic : Dynamic Programming

Important:

- Every substring is also a sub-sequence, but a Sub-sequence may or may not be a substring.



[MSQ]

#Q. Given a string S = "CABDABBCD".

Which of the following are valid sub-sequences?

A. BBD

B. BAC

C. CDCA

D. CADC

E. ADABD

A, B, D, E



Topic : Dynamic Programming

Longest Common Sub-Sequence:

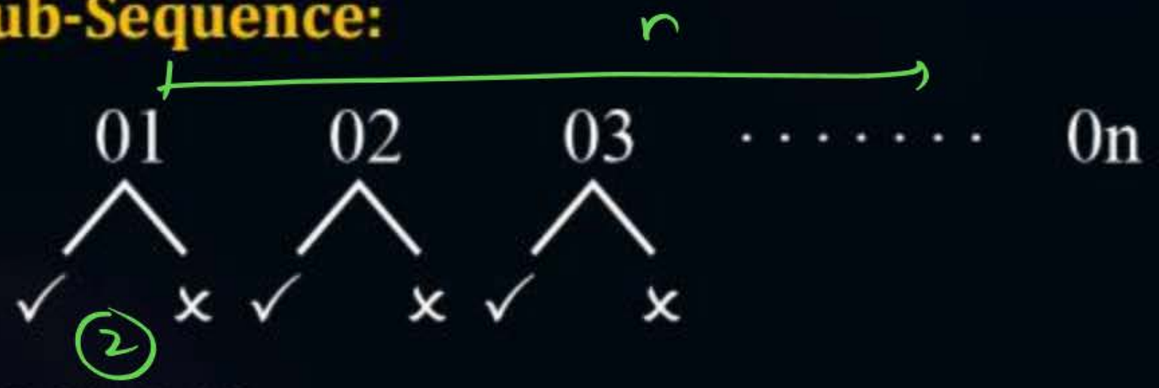
#Q. Given a string of length 'n' characters, the number of sub-sequences possible are?

→ 2^n

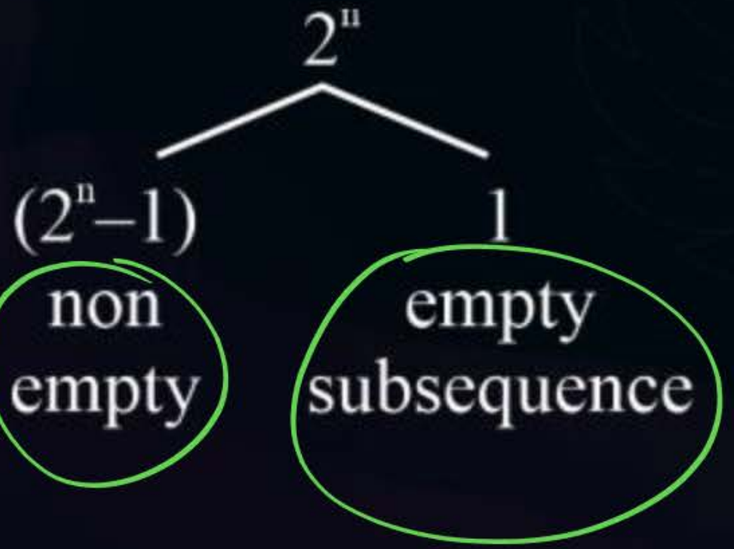


Topic : Dynamic Programming

Longest Common Sub-Sequence:



$\phi \rightarrow$ empty sub-sequence





Topic : Dynamic Programming

Brute Force/Enumeration: LCS

All Subsequence → A ✓

All Subsequence → B ✓

given A, B

- $O(2^n)$
- Hence, we need DP based approach.





Topic : Dynamic Programming

Longest Common Subsequence (DP based approach):

Common Subsequence:

- Given two strings X and Y of length n and m characters, a subsequence that is common to both X and Y is known as a common sub-sequence.



Topic : Dynamic Programming

Example:

LCS

#Q. X = "ABCD", Y = "BDC"

len(LCS) = 2

Which sub-sequences are common to both?

1) A ✗
2) B ✓
3) BC ✓
4) BD ✓
5) BCD ✗
6) DC ✗
7) BDC ✗

→ Common



Topic : Dynamic Programming

LCS Problem Statement:

#Q. Given strings 'A' and 'B' of length (Characters) 'n' & 'm' respectively. The LCS problem is to determine the subsequence that is of the longest length among all the subsequences that are common to both 'A' & 'B'?





Topic : Dynamic Programming

Example:

S1 = "ABCBDAB"

S2 = "BDCABA"

Longest LCS = 4



Topic : Dynamic Programming

Common Subsequences:

AB → 2

BCA → 3

BCB → 3

BCBA → 4

BCAB → 4

- Multiple longest common subsequences are possible.
- But LCS length is unique → 4



Topic : Dynamic Programming

DP based solution to LCS:

- Let 'i' and 'j' be indices in the strings 'X' and 'Y' respectively, such that.

$X = [x_1, x_2, x_3, \ldots, x_n]$

$Y = [y_1, y_2, y_3, \ldots, y_m]$



Topic : Dynamic Programming

- Let L(i, j) represent the length of the common sub-sequence of the strings X and Y.

$L(i, j) = 1 + L(i-1, j-1)$; if $X[i] == Y[j]$

$= \max\{L(i-1, j), L(i, j-1)\}$; if $X[i] \neq Y[j]$

$X \Rightarrow x_1 x_2 \cdots x_m$ (i)

$Y \Rightarrow y_1 y_2 \cdots y_n$ (j)

$L(i, j)$



Topic : Dynamic Programming

Initialization:

- L[-1, j] = 0,

  1st string is exhausted

  if i = –1; j = –1, 0, 1, 2, …, m–1

- L[i, –1] = 0,

  2nd string is exhausted

  if j = –1; i = –1, 0, 1, 2, …, n–1



Topic : Dynamic Programming

Example:

Case1: X[i] = Y[j]

X = "ABBCDCCABA"

Y = "CBDADCBDAA"

n

m

1+( )

$$L(i,j) = 1 + L(i-1, j-1)$$



Topic : Dynamic Programming

Example:

Case2: $X[i] \neq Y[j]$

X = "ABCDBBCCADB"

Y = "BDBCAAABDCD"

$\max\{L(i, j-1), L(i-1, j)\}$



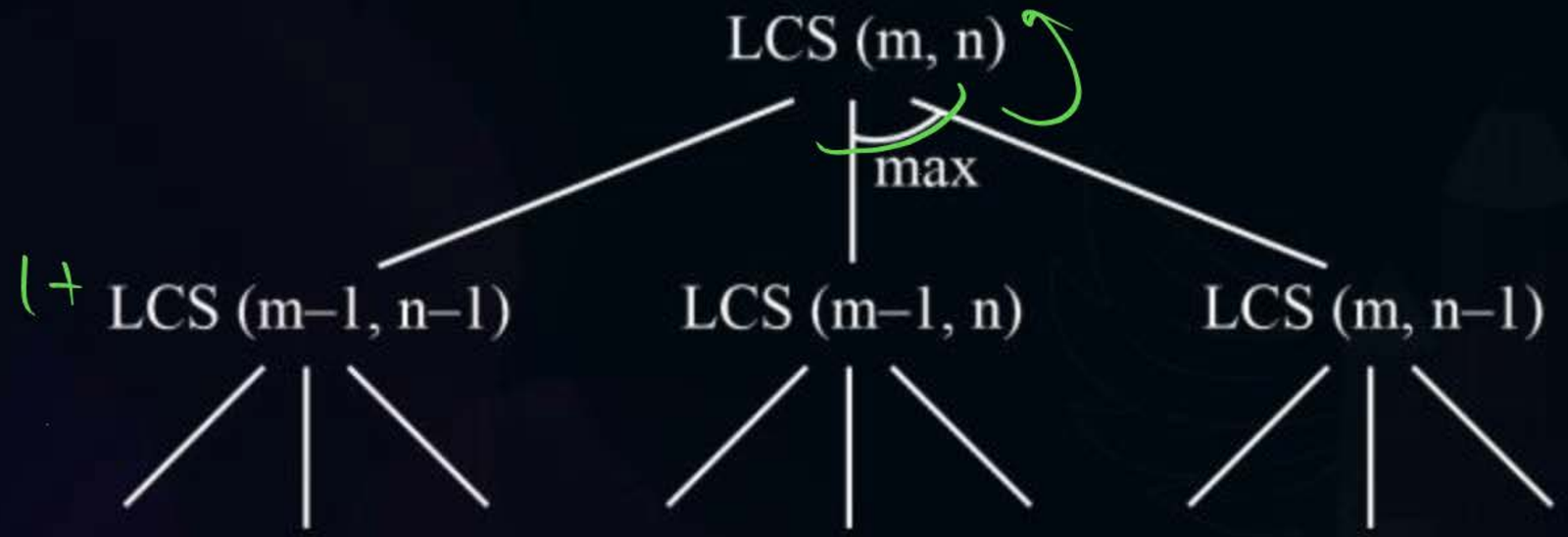
Topic : Dynamic Programming

Overlapping Subproblems:

$|X| = n, |Y| = m$

$|X| = 4$

$|Y| = 5$

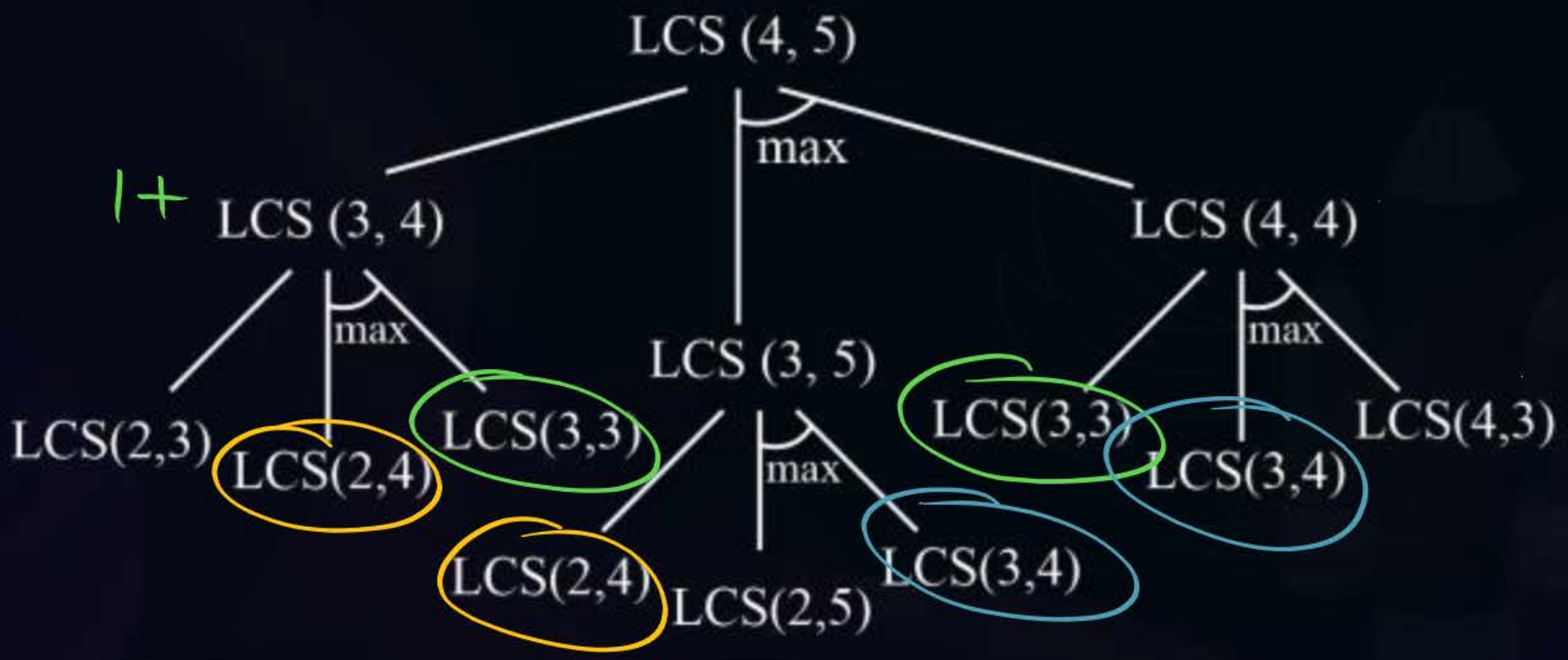




Topic : Dynamic Programming

Overlapping Subproblems:

|X| = 4, |Y| = 5





2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

0/1 Knap

Floyd Warshall

LCS



THANK - YOU